

Lab 4: Hardware Data Prefetcher Design and Analysis

Introduction

In this lab, I implement a GHB-based stride hardware data prefetcher. In a later step I improve the design by making it aware of system memory bandwidth constraints. Finally I create my own hybrid prefetcher, which chooses between a best-offset-prefetcher (Michaud 2016) and the previous strided GHB prefetcher.

Warm Up: Running the No-prefetching Baseline

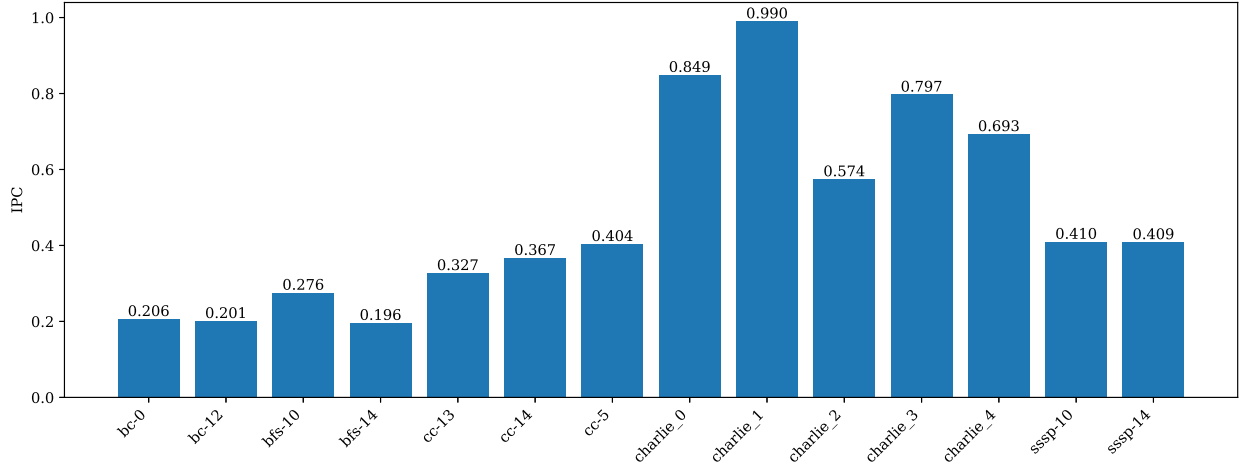


Figure 1: Baseline performance of the system without an L2C prefetcher on real-world CPU traces

The hardware prefetcher designs are evaluated on CPU traces collected from fourteen workloads sourced from graph analytics benchmarks and data-center workloads. Each workload was run for 50 million cycles following 10 million cycles of initial warmup using the Champsim microarchitectural simulator (Gober et al. 2022). The simulated system has a single out-of-order core and a single channel of DRAM running at a data rate of 4800 MT/s (“full BW”).

In fig. 1 is a plot of the instructions-per-cycle (IPC) for each workload. The `charlie_1` trace resulted in the highest IPC value, while the `bfs-14` trace got the lowest IPC value.

Task 1/4: Implementing GHB-based Stride Prefetcher

Implementation

In Task 1 I implemented a GHB-based stride prefetcher (Nesbit and Smith 2004) for the L2C as closely as possible to the original specification. I took special care to accurately replicate the bit-width of tags in the Index Table (IT) and pointers in the Global History Buffer (GHB). However in my own testing, I found that the IPC results for the exact replication are identical to those with a more relaxed replication of the paper with wider bit-widths (tbl. 1). First, this shows that the hardware budget proposed in the paper provides performance equal to an expensive prefetcher, but with a fraction of the resources. Second, this means that my replication of the original prefetcher isn't sensitive to the specific sizings of the table entries.

Table 1: Per-trace IPC for GHB PC/CS with 20b and 56b wide IT tags (1 cpu, full BW)

trace	1	2	3	4	5	6	7	8	9	10	11	12	13	14
20b IPC	0.229	0.221	0.362	0.293	0.415	0.476	0.532	0.864	1.014	0.582	0.809	0.704	0.548	0.554
56b IPC	0.229	0.221	0.362	0.293	0.415	0.476	0.532	0.864	1.014	0.582	0.809	0.704	0.548	0.554

The per-workload IPC speedups, prefetching accuracies and prefetches per thousand instructions (PPKI) caused by the strided GHB prefetcher in the L2C are plotted in fig. 2. The IPC speedup is simply the ratio of the IPC measured with the prefetcher and the IPC without the prefetcher. Next, the accuracy is defined as the ratio of prefetches reported by Champsim logs as being “useful” (`l2c_prefetch_useful`) and the total number of issued prefetches (`l2c_prefetch_issued`).

Results

The utility of the prefetcher is highly heterogenous for the different types of workloads. As explained in the following paragraphs, this indicates that the strided GHB prefetcher is only useful for workloads with access patterns that play to its strengths. Notice that the IPC speedup is always greater than 1, implying that the prefetcher gains a strict performance increase.

GAP Workloads `bfs-14` was the workload with the largest IPC speedup. It was also the workload with the largest amount of prefetches issued, at the highest accuracy. The large number of prefetches issued indicate that the GAP workloads (graph analytics) can benefit a lot from a strided GHB prefetcher, considering they are most likely traversing neighboring nodes lists.

Charlie Workloads `charlie_2` got the lowest IPC speedup of all workloads. With the exception of `charlie_3`, very few prefetches were issued for this group of workloads, leading to very low IPC speedups overall. For `charlie_3`, the number of issued prefetches was much higher, but with very poor accuracy. This indicates that the charlie workloads (sourced from Google data centers) feature cache accesses that don't suit a constant-stride prefetcher's profile. Possibly these workloads resemble pointer-chasing or random accesses.

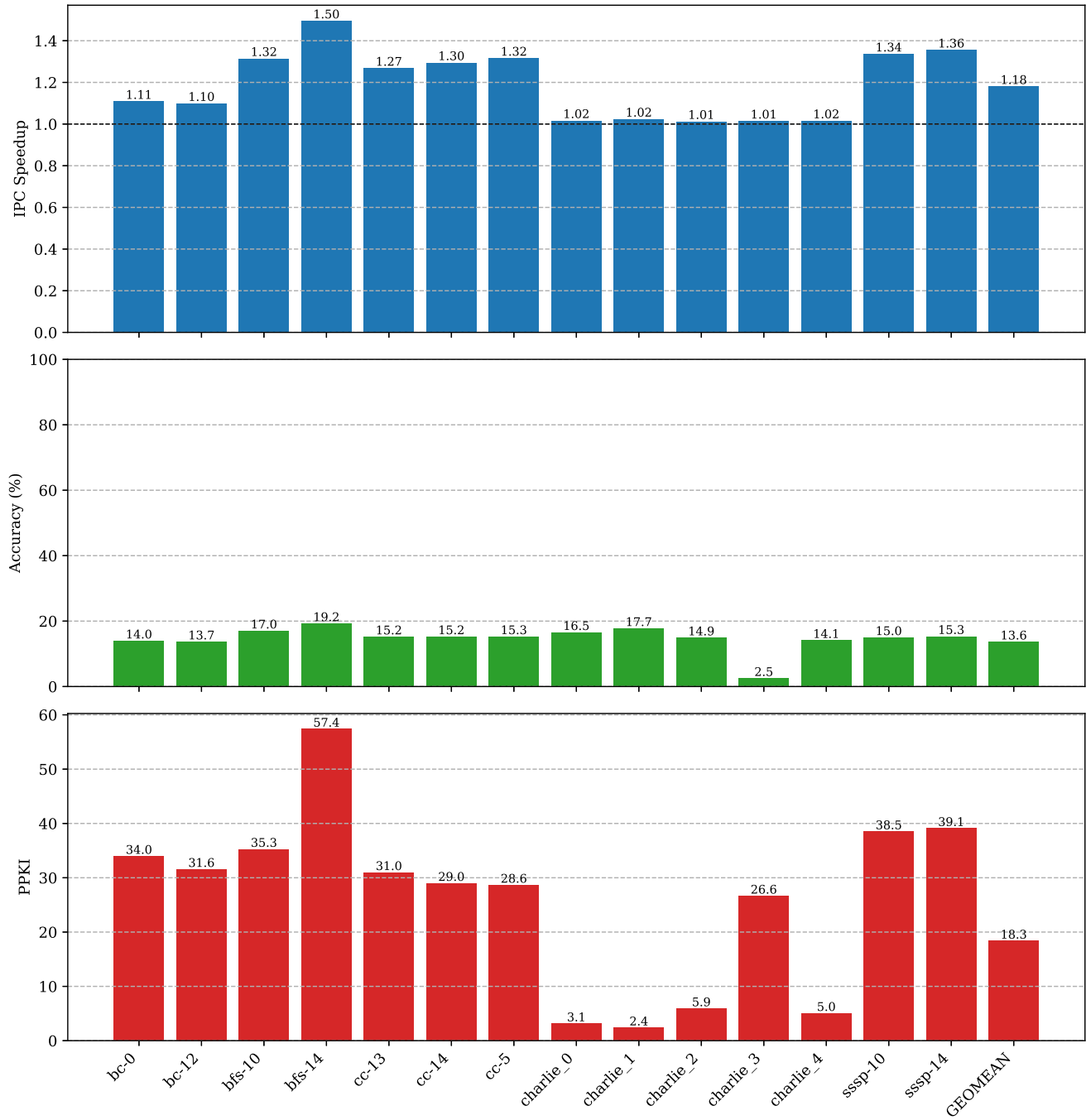


Figure 2: Task 1: IPC speedup, prefetch accuracy, and prefetches per thousand instructions (PPKI) of strided GHB prefetcher (1 CPU, full BW)

Task 2/4: Prefetching with Limited Main Memory Bandwidth

In this task, I investigate the effects of limited main memory bandwidth on the strided GHB prefetcher. The system with limited main memory bandwidth (“limited BW”) is identical to the system with full BW except that the main memory bandwidth is 800 MT/s, a sixth of the full BW.

Results

The performance of the strided GHB prefetcher in a memory bandwidth limited system is plotted in fig. 3. There are a few observations to make.

- First, the IPC speedups measured in the limited BW system are strictly lower than those from the unconstrained full BW system.
- Next, the IPC speedup on the `charlie_3` workload is below 1, indicating a performance decrease. It achieves the lowest IPC speedup of all workloads. This is most likely due to the combinations of large PPKI, low accuracy and limited bandwidth availability. The largest IPC speedup is observed in the `bfs-14` workload.
- Finally, the largest IPC speedup decreases are on the `sssp-10` and `sssp-14` traces with `-0.104` and `-0.106` respectively. The smallest IPC speedup decrease is on the `cc-13` workload with `-0.002`.

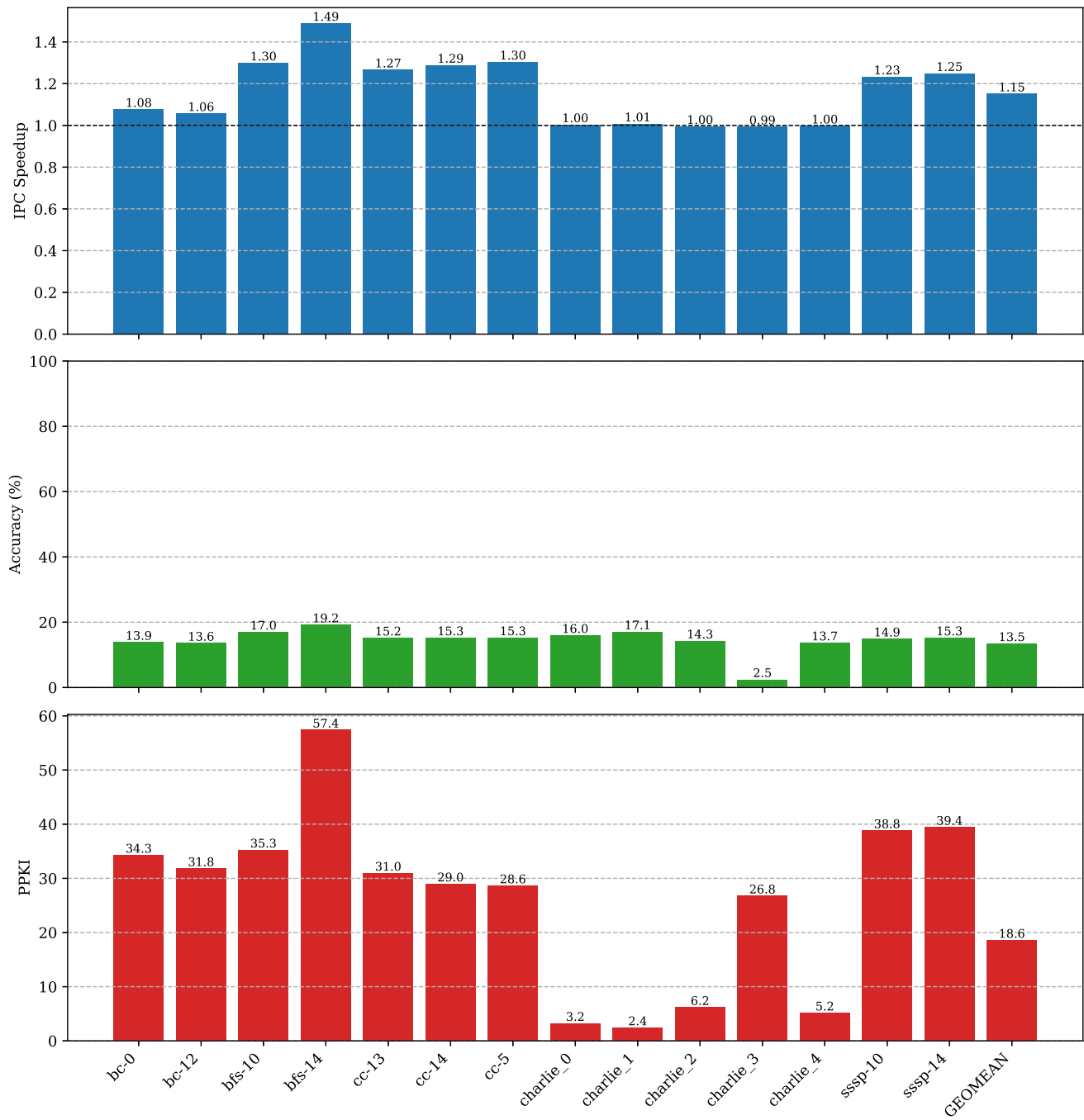


Figure 3: Task 2: IPC speedup, prefetch accuracy, and prefetches per thousand instructions (PPKI) of strided GHB prefetcher (1 CPU, limited BW)

Task 3/4: System-Aware Prefetcher Design

As suggested in the task description, the strided GHB prefetcher performs worse in a bandwidth-limited system because speculative prefetches consume valuable memory bandwidth. Task 3 therefore introduces system feedback: the prefetcher adaptively throttles its aggressiveness using a heuristic scheme described in the assignment.

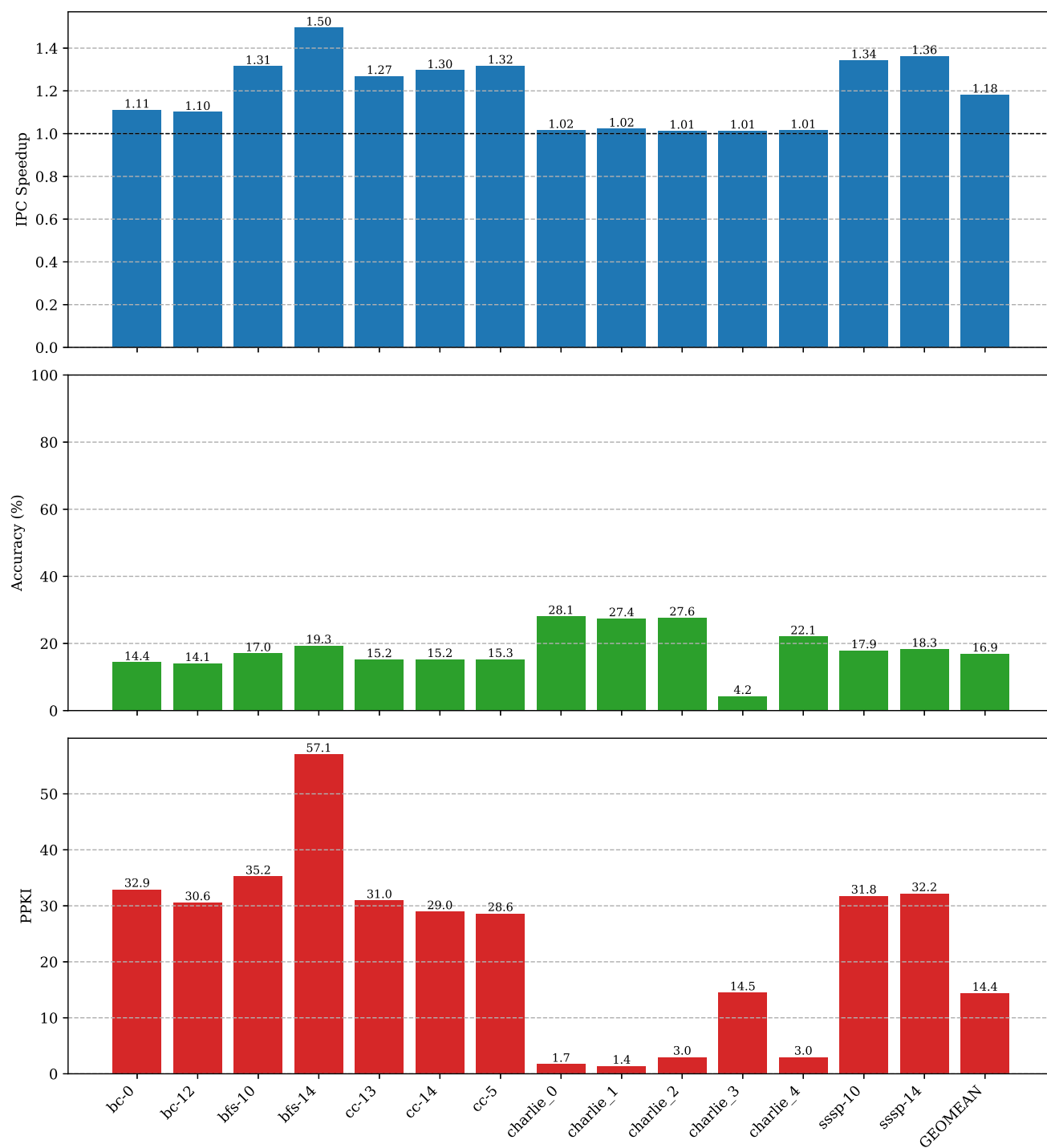
Results

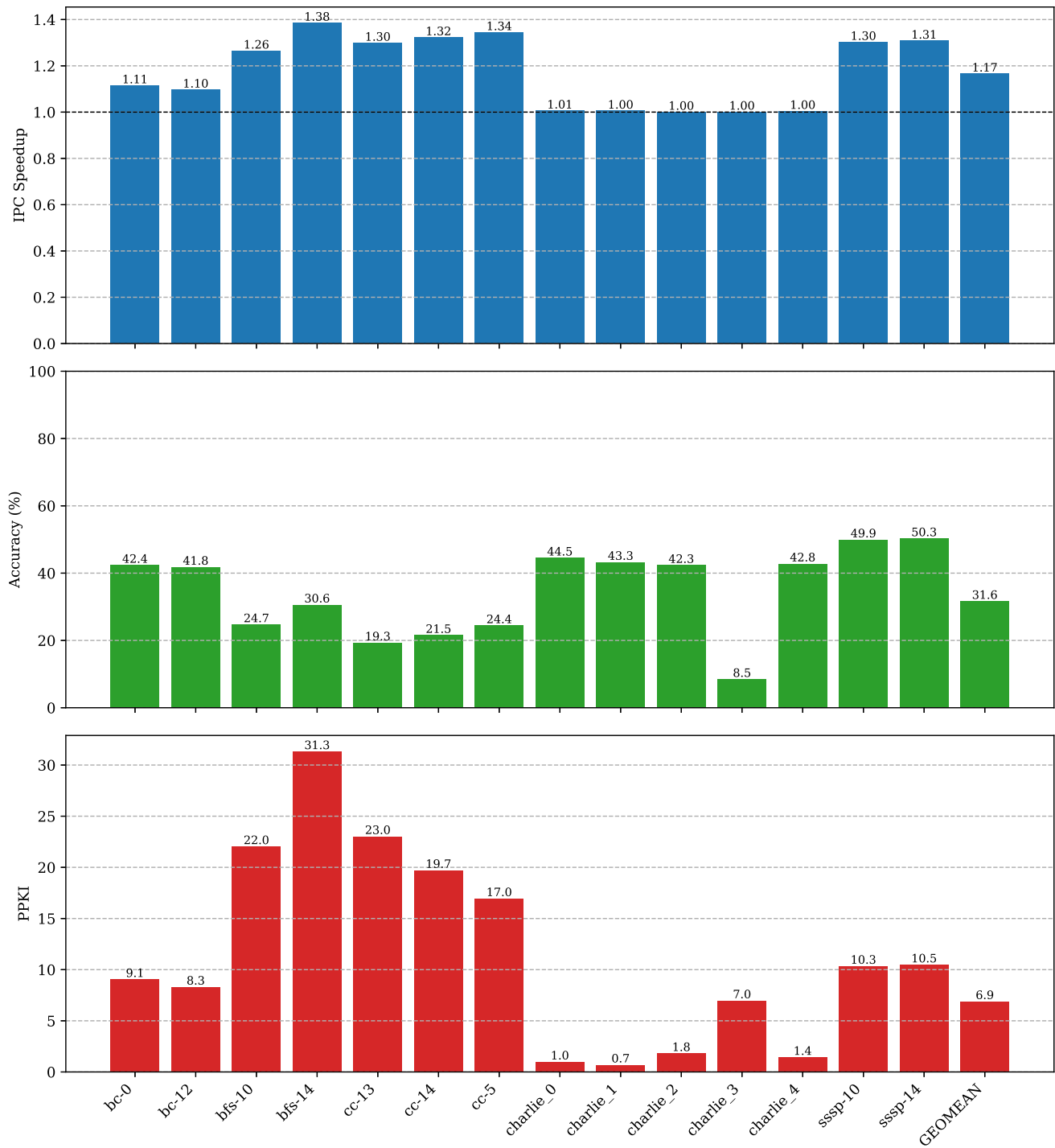
The performance of the system-aware strided GHB prefetcher in both the full-BW and limited-BW systems is shown in fig. 4 and fig. 5, respectively.

Full Bandwidth System In the full-BW system, the system-aware prefetcher achieves higher accuracy on most workloads while preserving the IPC speedups observed with the system-unaware prefetcher. In other words, it issues fewer pointless prefetches without sacrificing performance.

Limited Bandwidth System In the limited-BW system, the system-aware prefetcher reduces IPC speedups for `bfs-10` and `bfs-14`, but otherwise maintains or improves them relative to the system-unaware baseline. Overall, the geometric mean IPC speedup declines less than with the system-unaware prefetcher under bandwidth constraints.

Prefetch accuracy increases by more than 18% (geometric mean), reflecting stricter issuance and better selectivity. This is consistent with the lower PPKI observed: the system-aware prefetcher issues substantially fewer prefetches, conserving bandwidth for useful demand requests.

Figure 4: Task 3: System-Aware Strided GHB Prefetcher (1 CPU, *Full* BW)

Figure 5: Task 3: System-Aware Strided GHB Prefetcher (1 CPU, *Limited* BW)

Task 4/4: Design Your Own Prefetcher

In Task 4, I implemented a custom hybrid prefetcher designed to adapt to varying workload characteristics.

Design Rationale

My initial design was focused on the Best-Offset Prefetcher (BOP) (Michaud 2016), motivated by the observation that the strided GHB prefetcher struggled with the irregular access patterns of the “Charlie” workloads. As shown in fig. 6, the BOP outperforms the system-aware GHB on the Charlie traces and specific graph workloads like **bfs-10** and **bfs-14**. However, on the remaining graph analytics workloads, the strided GHB retains a substantial performance advantage.

Recognizing the complementary performance profiles of these two designs, I implemented a hybrid tournament prefetcher. This design dynamically selects between the BOP and the strided GHB based on their runtime accuracy.

Hybrid Architecture

The hybrid prefetcher maintains active instances of both the GHB and BOP engines. To determine which prefetcher is currently more effective, it employs a tournament selection mechanism governed by a “shadow table” and a saturation counter.

1. Tournament State The system state is tracked by a single integer counter ranging from 0 to 10, initialized to a neutral value of 5. The counter acts as a sliding scale of confidence: values toward 0 indicate a strong preference for the Best-Offset Prefetcher (BOP), while values toward 10 indicate a strong preference for the Strided GHB.

2. Shadow Table & Virtual Prefetching A “shadow table” is used to track the theoretical accuracy of each prefetcher. This table maps memory addresses to a creator ID (1=GHB, 2=BOP, 3=Both). On every cache access, both sub-prefetchers calculate their candidate addresses, but do not issue them directly. Instead, these “virtual prefetches” are inserted into the shadow table.

- **Fairness Mechanism:** To ensure a fair comparison, the shadow table is only updated on cache misses or known useful prefetches. This prevents the GHB, which generates candidates on nearly every access, from flooding the table and drowning out the BOP, which triggers less frequently.
- **Collision Handling:** If a prefetcher suggests an address already present in the table (created by the rival engine), the entry is marked as “Both”. This ensures that if both engines correctly predict a line, the system remains neutral rather than arbitrarily favoring one.
- **Table Management:** The table has a limited size (256 entries). If the table fills up, a Least-Recently-Used (LRU) policy evicts the oldest entries to make room for new predictions.

3. Score Update When a demand request hits an entry in the shadow table, it indicates a successful prediction. The tournament counter is updated based on the creator of that entry:

- **GHB Correct:** Increment counter (shift toward 10).
- **BOP Correct:** Decrement counter (shift toward 0).
- **Both Correct:** The counter remains unchanged. The entry is immediately removed from the table after a hit to prevent double-counting.

4. Arbitration and Issuance Finally, the system decides which candidates to actually issue to the L2C based on the tournament counter:

- Counter ≥ 6 (Lean GHB): Only GHB candidates are issued.
- Counter ≤ 4 (Lean BOP): Only BOP candidates are issued.
- Counter $== 5$ (Neutral): Candidates from both engines are issued. This “neutral zone” allows the system to explore both strategies when confidence is low or when the workload transitions.

Results

The hybrid prefetcher demonstrates robust performance stability across the benchmark suite. As shown in fig. 6 and fig. 7, the hybrid design generally tracks the maximum speedup of its constituent prefetchers.

- Overall Improvement: The geometric mean IPC speedup is 1% higher than the system-aware GHB baseline.
- Adaptability: It successfully captures the high performance of BOP on Charlie workloads while retaining GHB’s advantage on standard graph algorithms.
- Limitations: A notable regression occurs on **bfs-10** and **bfs-14**, where the hybrid performs worse than the standalone BOP.

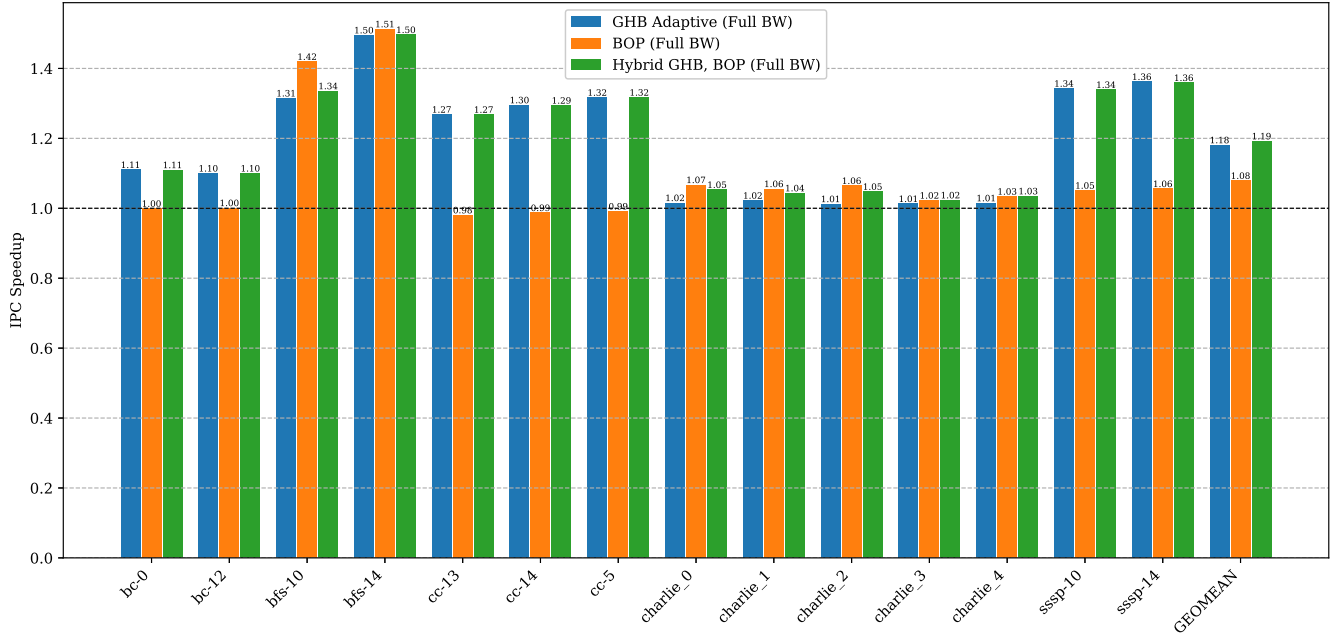


Figure 6: Task 4: Hybrid prefetcher chooses the better performing of either BOP or GHB (1 CPU, *Full BW*)

Future Work

The analysis of the hybrid prefetcher suggests that its performance ceiling was limited not by the selection logic, but by the similarity between the chosen sub-prefetchers. While the tournament mechanism proved effective at dynamically selecting the better engine, the Best-Offset Prefetcher (BOP) did not offer a sufficiently large performance advantage over the strided GHB to drive significant gains.

Future work should focus on replacing the BOP with a prefetcher that exhibits a more orthogonal performance profile to the strided GHB. Since the GHB handles regular strided patterns well, a complementary design—such as a temporal or spatial prefetcher would be better suited to handle the irregular access patterns seen in the “Charlie” workloads. By pairing the GHB with a prefetcher that targets completely different access behaviors, the hybrid system could achieve much higher aggregate performance.

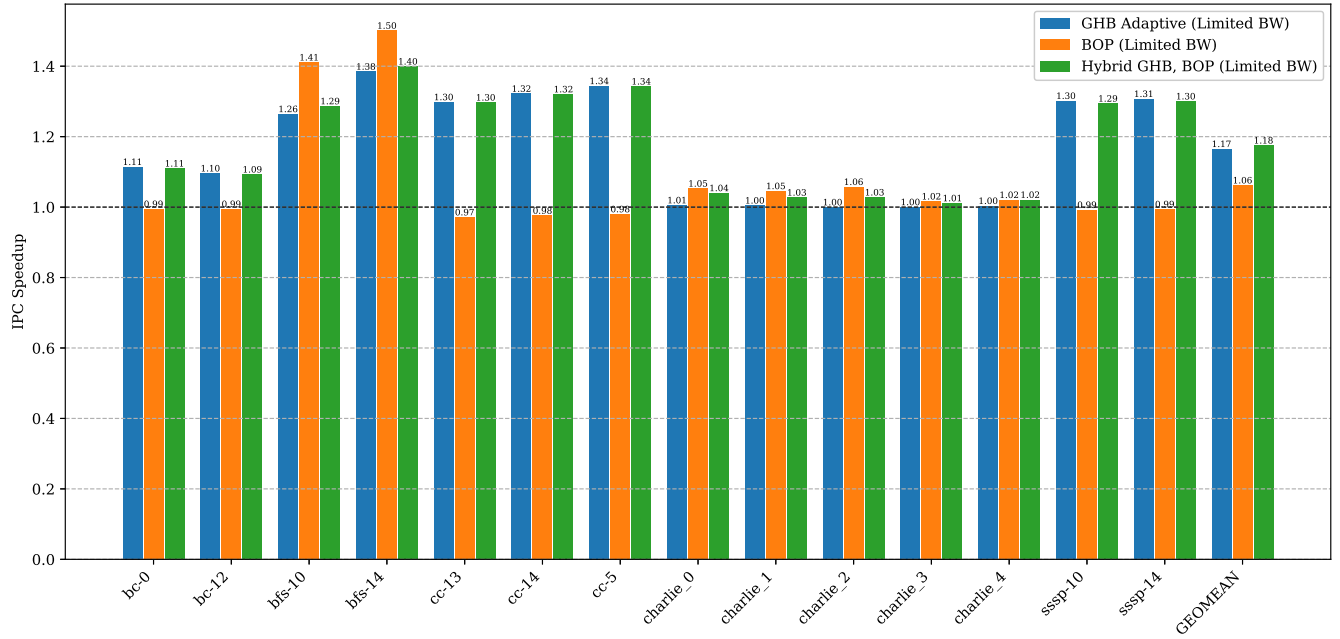


Figure 7: Task 4: Hybrid prefetcher chooses the better performing of either BOP or GHB (1 CPU, *Limited BW*)

Bonus Task: Comparison Against a State-Of-The-Art Prefetcher

In the bonus task, I compare Pythia (Bera et al. 2021), a state-of-the-art prefetcher against the other prefetchers I designed in this lab.

The performance of Pythia compared to the System-Aware GHB and the BOP/GHB Hybrid designs is plotted in fig. 8 and fig. 9.

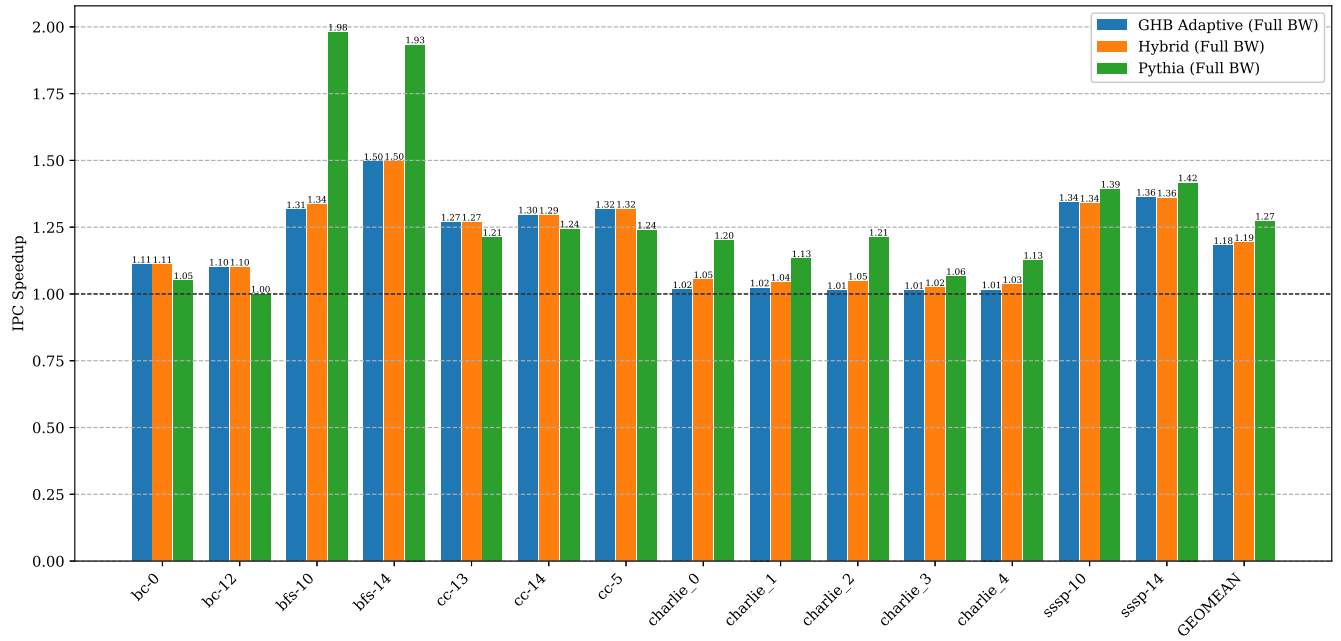
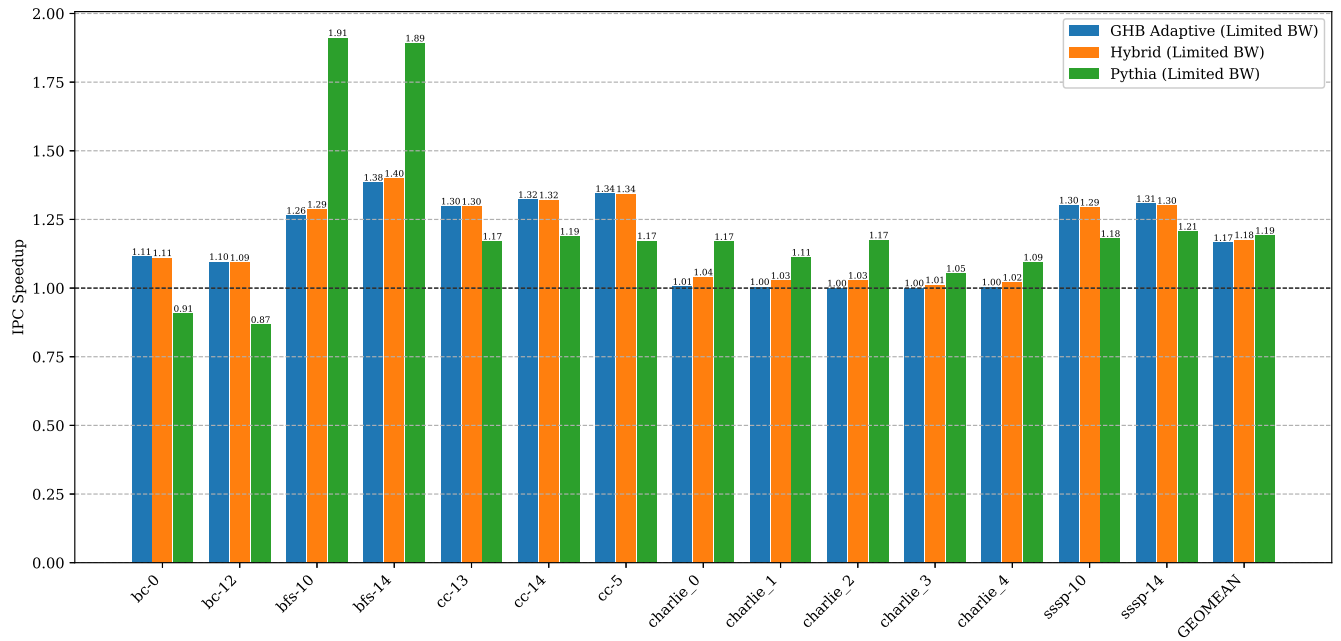
Results

Full Bandwidth System In the unconstrained bandwidth configuration, Pythia demonstrates a significant performance advantage over the heuristic-based prefetchers, achieving a geometric mean speedup of 1.27 compared to 1.18 for the Hybrid design.

- **Complex Patterns:** Pythia excels in workloads with irregular or complex access patterns. notably the **charlie** traces and the **bfs** workloads. In **bfs-10** and **bfs-14**, Pythia achieves speedups over 1.9x, vastly outperforming the ~1.5x achieved by the GHB/BOP designs. This suggests that the Reinforcement Learning (RL) agent successfully learns access patterns that are not strictly strided or offset-based.
- **Regular Patterns:** In highly regular patterns like **bc**, Pythia performs slightly worse than the dedicated stride/offset prefetchers.

Limited Bandwidth System

- While Pythia retains its dominance in the **bfs** and **charlie** workloads, it suffers severe degradation in the **bc** traces, dropping below a speedup of 1.0 (indicating performance degradation). This implies that Pythia acts too aggressively for the constrained bandwidth, displacing demand requests.
- Comparison to System-Awareness: Unlike the System-Aware GHB designed in Task 3, which explicitly throttles its degree based on bandwidth usage, Pythia's default reward function may not penalize bandwidth consumption heavily enough.

Figure 8: Bonus Task: IPC Speedup Comparison (1 CPU, *full* BW)Figure 9: Bonus Task: IPC Speedup Comparison (1 CPU, *Limited* BW)

Citations

- Bera, Rahul, Konstantinos Kanellopoulos, Anant V. Nori, Taha Shahroodi, Sreenivas Subramoney, and Onur Mutlu. 2021. “Pythia: A Customizable Hardware Prefetching Framework Using Online Reinforcement Learning.” *CoRR* abs/2109.12021. <https://arxiv.org/abs/2109.12021>.
- Gober, Nathan, Gino Chacon, Lei Wang, et al. 2022. *The Championship Simulator: Architectural Simulation for Education and Competition*. <https://arxiv.org/abs/2210.14324>.
- Michaud, Pierre. 2016. “Best-Offset Hardware Prefetching.” *2016 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 469–80. <https://doi.org/10.1109/HPCA.2016.7446087>.
- Nesbit, K. J., and J. E. Smith. 2004. “Data Cache Prefetching Using a Global History Buffer.” *10th International Symposium on High Performance Computer Architecture (HPCA '04)*, 96–96. <https://doi.org/10.1109/HPCA.2004.10030>.